Task 3

Automated Infrastructure Scripting & Secrets Hygiene

Elevvo Pathways | Cloud & DevOps Engineering Internship

Prepared by: Dorcas Torshie Sai

Environment: Ubuntu on VirtualBox; Bash, Python 3, cron, Git

GitHub repository: github.com/dorcassai-ux/Task3-Secrets_Hygiene

1. Overview

This task automates a routine maintenance job: a daily backup of a web directory, plus a log-checking script. Both are scheduled or run without manual work, and the whole project is tracked in Git without exposing any secrets.

Requirement	Deliverable	Status
Bash script that compresses a directory into a date-tagged .tar.gz, using set -euo pipefail	backup.sh	Done
Cron entry running the script daily at midnight	cron.txt (installed with crontab)	Done
Python script that counts "ERROR" lines and reads a token from the environment	log_parser.py	Done
Secrets hygiene: .gitignore and a clean commit history	.gitignore, Git history	Done

2. Project Structure

```
task3-backup/
  backup.sh        Bash backup script
  log_parser.py    Python log parser
  cron.txt         Crontab entry (daily at midnight)
  .gitignore       Keeps secrets, logs and archives out of Git
  README.md        Project documentation
  test-folder/     Small folder used to test the backup
```

3. The Backup Script (Bash)

```
#!/bin/bash
set -euo pipefail

SOURCE_DIR="${1:-/var/www/html}"
BACKUP_DIR="${HOME}/backups"
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)

mkdir -p "$BACKUP_DIR"
tar -czf "$BACKUP_DIR/backup-$TIMESTAMP.tar.gz" "$SOURCE_DIR"
echo "Backup created: $BACKUP_DIR/backup-$TIMESTAMP.tar.gz"
```

How it works

- **#!/bin/bash** tells Linux to run the file with Bash.
- **set -euo pipefail** is the fail-fast safety line. -e stops the script when a command fails, -u stops it when a variable is not set, and -o pipefail makes a failure inside a pipeline count as a failure.
- **SOURCE_DIR="${1:-/var/www/html}"** uses the first argument as the folder to back up, or /var/www/html when no argument is given.
- **TIMESTAMP=$(date ...)** stores the current date and time, so every archive has a unique, sortable name.
- **mkdir -p "$BACKUP_DIR"** creates the backup folder if it does not exist.
- **tar -czf** creates (c) a gzip-compressed (z) archive file (f) of the source folder.
- Every variable is quoted ("$VAR") so that names containing spaces cannot break the command.

4. Scheduling with Cron

The exact crontab entry is stored in cron.txt:

```
0 0 * * * /bin/bash /home/dorcas-sai/task3-backup/backup.sh >> /home/dorcas-sai/task3-backup/backup.log 2>&1
```

Part	Meaning	Reason
0 0 * * *	Minute 0, hour 0, every day	Runs daily at midnight
/bin/bash	Full path to Bash	Cron runs with a minimal environment, so absolute paths are used
/home/dorcas-sai/.../backup.sh	Full path to the script	Relative paths fail under cron
>> backup.log	Append normal output to a log file	Creates an audit trail
2>&1	Send errors to the same place as output	Failures are recorded for debugging

The entry is installed with:

```
crontab cron.txt
crontab -l          # lists the installed schedule
```

5. The Log Parser (Python)

```
import os

api_key = os.environ.get("API_KEY")
if not api_key:
    raise SystemExit("API_KEY not set in environment")

count = 0
try:
    with open("app.log", "r") as f:
        for line in f:
            if "ERROR" in line:
                count += 1
    print(f"Total ERRORs found: {count}")
except FileNotFoundError:
    print("app.log not found")
```

- The simulated API token is read from the environment with os.environ.get. It is never written in the code.
- If the variable is missing, the script stops with a clear message instead of running without credentials.
- try/except handles a missing log file cleanly instead of crashing with a traceback.

Running it:

```
python3 log_parser.py              # API_KEY not set in environment
export API_KEY="my-fake-key-123"   # a fake key for practice only
python3 log_parser.py              # Total ERRORs found: 3
```

6. Secrets Hygiene

The .gitignore file keeps private and generated files out of version control:

Entry	Why it is ignored
.env	Local settings and secrets
*.tar.gz	Backup archives are large and may contain private data
*.log	Logs can contain sensitive information
backup.log	The cron output log
__pycache__/	Python temporary files

No password or token appears anywhere in the code or the commit history. The only credential, API_KEY, lives in the environment of the running shell. Environment variables keep configuration separate from code, so the same script can run in any environment without changes.

7. Version Control

```
git init
git add .
git commit -m "Add backup script and log parser"
git branch -M main
git remote add origin https://github.com/dorcassai-ux/Task3-Secrets_Hygiene.git
git push -u origin main
```

git status was checked before the first commit to confirm that only the intended files were staged:

```
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ls
BACKUP_DIR  README.md  app.log  backup.sh  cron.txt  log_parser.py  test-folder
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/dorcas-sai/task3-backup/.git/
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        backup.sh
        cron.txt
        log_parser.py
        test-folder/
```

Figure 1: git status before the first commit. Only the project files appear, and no secrets.

8. Testing and Evidence

The backup script was tested on a small folder. The first runs failed because of a syntax error, which was fixed (see Section 9). The final run created the archive.

```
dorcas-sai@dorcas-sai-VirtualBox:~$ cd ~
dorcas-sai@dorcas-sai-VirtualBox:~$ mkdir task3-backup
dorcas-sai@dorcas-sai-VirtualBox:~$ cd task3-backup
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ pwd
/home/dorcas-sai/task3-backup
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ nano backup.sh
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ chmod +x backup.sh
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ls -l
total 4
-rwxrwxr-x 1 dorcas-sai dorcas-sai 274 Nov  9 18:10 backup.sh
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ mkdir -p test-folder
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ echo "hello" > test-folder/myfile.txt
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ./backup.sh test-folder
./backup.sh: line 10: unexpected EOF while looking for matching `"'
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ nano backup.sh
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ./backup.sh test-folder
./backup.sh: line 10: unexpected EOF while looking for matching `"'
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ nano backup.sh
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ./backup.sh test-folder
Backup created: /home/dorcas-sai/backups/backup_2026-11-09_20-22-33.tar.gz
```

Figure 2: Creating and testing backup.sh. The syntax error is fixed and the backup is created.

Date-stamped archives in the backup folder, and the Python parser run before and after exporting the key:

```
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ls -lh ~/backup.sh
ls: cannot access '/home/dorcas-sai/backup.sh': No such file or directory
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ls -lh ~/backups
total 16K
-rw-rw-r-- 1 dorcas-sai dorcas-sai 172 Nov  9 20:18 backup-2026-11-09_20-18-06.tar.gz
-rw-rw-r-- 1 dorcas-sai dorcas-sai 172 Nov  9 20:21 backup-2026-11-09_20-21-45.tar.gz
-rw-rw-r-- 1 dorcas-sai dorcas-sai 172 Nov  9 20:22 backup-2026-11-09_20-22-33.tar.gz
-rw-rw-r-- 1 dorcas-sai dorcas-sai 202 Oct 30 19:06 backup_2026-10-30_19-06-14.tar.gz
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ pwd
/home/dorcas-sai/task3-backup
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ nano app.log
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ nano log_parser.py
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ python3 log_parser.py
API_KEY not set in environment
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ export API_KEY="my-fake-key-123"
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ python3 log_parser.py
Total ERRORs found: 3
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ nano cron.txt
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ nano .gitignore
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ls -la
total 36
drwxrwxr-x  4 dorcas-sai dorcas-sai 4096 Nov  9 21:36 .
drwxr-x--- 37 dorcas-sai dorcas-sai 4096 Nov  9 17:59 ..
-rw-rw-r--  1 dorcas-sai dorcas-sai   45 Nov  9 21:36 .gitignore
drwxrwxr-x  2 dorcas-sai dorcas-sai 4096 Nov  9 20:18 BACKUP_DIR
-rw-rw-r--  1 dorcas-sai dorcas-sai  136 Nov  9 20:30 app.log
-rwxrwxr-x  1 dorcas-sai dorcas-sai  275 Nov  9 20:22 backup.sh
-rw-rw-r--  1 dorcas-sai dorcas-sai  104 Nov  9 21:35 cron.txt
-rw-rw-r--  1 dorcas-sai dorcas-sai  351 Nov  9 20:31 log_parser.py
drwxrwxr-x  2 dorcas-sai dorcas-sai 4096 Nov  9 20:17 test-folder
```

Figure 3: Backup archives listed, log_parser.py stopping without API_KEY, then reporting 3 errors once it is set.

After the fixes, the script ran successfully. The exact cron command was also run by hand to prove that the scheduled line works. Its output is appended to backup.log. The tar notice about removing the leading slash is a normal warning, not an error, and it is captured in the log because of 2>&1.

```
dorcas-sai@dorcas-sai-VirtualBox:~/task3-backup$ ./backup.sh test-folder
Backup created: /home/dorcas-sai/backups/backup-2026-11-10_06-21-49.tar.gz
```

Figure 4: The fixed backup.sh creates a new date-stamped archive.

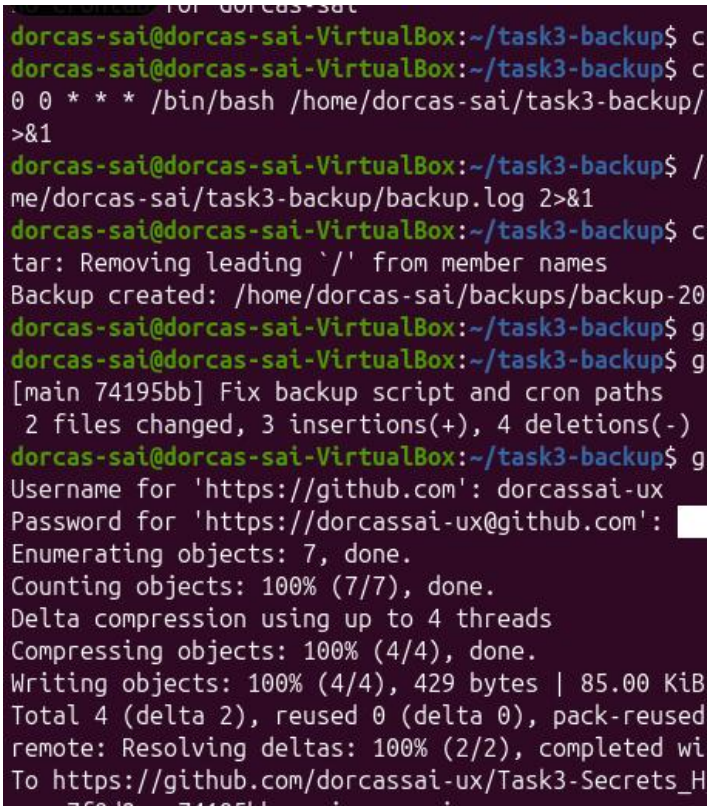


Figure 5: crontab -l shows the installed midnight entry, backup.log records the run, and the fixes are committed and pushed to GitHub.

9. Troubleshooting Log

Problem	Cause	Fix
unexpected EOF while looking for matching quote (line 10)	A missing closing quote in the script	Corrected the quote in nano and ran it again
mkdir created a folder named BACKUP_DIR	The variable was written without the $ sign	Changed it to "$BACKUP_DIR" and removed the stray folder
Default folder /var/ww/html does not exist	Typing mistake in the path	Corrected to /var/www/html
crontab -l: no crontab	cron.txt was only a text file and had not been installed	Installed it with crontab cron.txt
Cron paths used the wrong user and folder name	Path copied from another system	Rewrote the entry with the real /home/dorcas-sai/task3-backup paths
API_KEY not set in environment	The variable was not exported	Ran export API_KEY=... before the script
First push went to a wrongly named repository	Wrong remote URL	Fixed it with git remote set-url origin

10. What I Learned

- Fail-fast Bash (set -euo pipefail) prevents silent, cascading failures.
- Cron runs in a minimal environment, so scripts and commands need absolute paths and logged output.
- Secrets belong in environment variables and are kept out of Git with .gitignore.
- A missing $ or a one-letter typo can change what a script does, so every script needs a test run.

- Reading error messages carefully is the fastest way to find the problem.